Penn State University
Great Valley Campus

Engineering Division

Data Specification for
DAAN 825

11/16/2025

Modified from template at www.dpw.state.pa.us

Table of Contents

Project Summary **3**
Requirements Definition **3**
Document Change Log **4**
2. Architecture Design **5**
2.1 Relational Data Warehouse 5
Tables schemas 5
Examples of values 5
2.2 Hadoop Implementation 8
2.3 Reflective analysis of using a data warehouse vs Hadoop. 8
3. Data Preparation **8**
3.1 Relational Data Warehouse Implementation 8
3.2 Hadoop Implementation 11
3.3 Reflective analysis of data preparation in relational data warehouse vs Hadoop. 13
4. Reporting System **14**
4.1 Relational Data Warehouse Implementation 14
4.2 Hadoop Implementation 16
4.3 Reflective analysis of result in relational data warehouse vs Hadoop. 19
Conclusions **19**

DAAN 825 Project 1
Joshua Martin

PROJECT SUMMARY

The following is a bullet-point summary of the objective and scope of the project as well as the source of the data.

A. Objectives

Determine what conclusions can be drawn from the information on the types of contributors and their trends in the first 3 months of 2023 for the states of PA, TX, FL, and GA.

B. Scope

Using the data from the FEC website provided in the OneDrive folder linked on the Canvas course, and using DBeaver as a GUI interface for SQL, the data is from the first 3 months of 2023 for PA, TX, FL, and GA.

C. References

The FEC.gov website is the sole source of data for this project. Code and procedures from class materials is referenced and utilized throughout the project as well.

REQUIREMENTS DEFINITION

A. Goals

Utilize DBeaver and Hadoop to create and structure a database from publicly available FTC filings which is then used in KNIME to visualize the data and draw conclusions about information of interest.

B. Usability Requirements

Requirements for recreation of the data found in this report include simply internet access and knowledge of how to use DBeaver, Hadoop, and KNIME software packages.

C. System Security Requirements

All data is publicly accessible and all software used is also publicly accessible and following the methods would allow recreation of the charts.

D. Business Questions

What are trends in contributions in these states and what are the differences in the demographics of the kinds and frequency of donations in these different states? Are there

differences in per capita information, or are there specific committees which disproportionately receive donations, and do these committees depend on state?

E. Data Requirements

In order to answer the Business questions above, the data will be loaded into a database format where it will be structured for easy use and then exported to KNIME for visualization purposes to allow for not only specific answers but also context to be shown from the data.

F. Design Constraints

I am not currently aware of any significant restraints based on the current design of the database architecture beyond the natural limitations and potential errors in the underlying data.

DOCUMENT CHANGE LOG

Change Date	Version	CR #	Change Description	Author and Organization
09/28/25	1.0		Initial creation.	Joshua Martin
10/19/25	1.01		Final part 1 submission	Joshua Martin
11/16/25	1.2		Part 2 draft	Joshua Martin
12/12/25	1.3		Final Submission	Joshua Martin

2. ARCHITECTURE DESIGN

2.1 Relational Data Warehouse

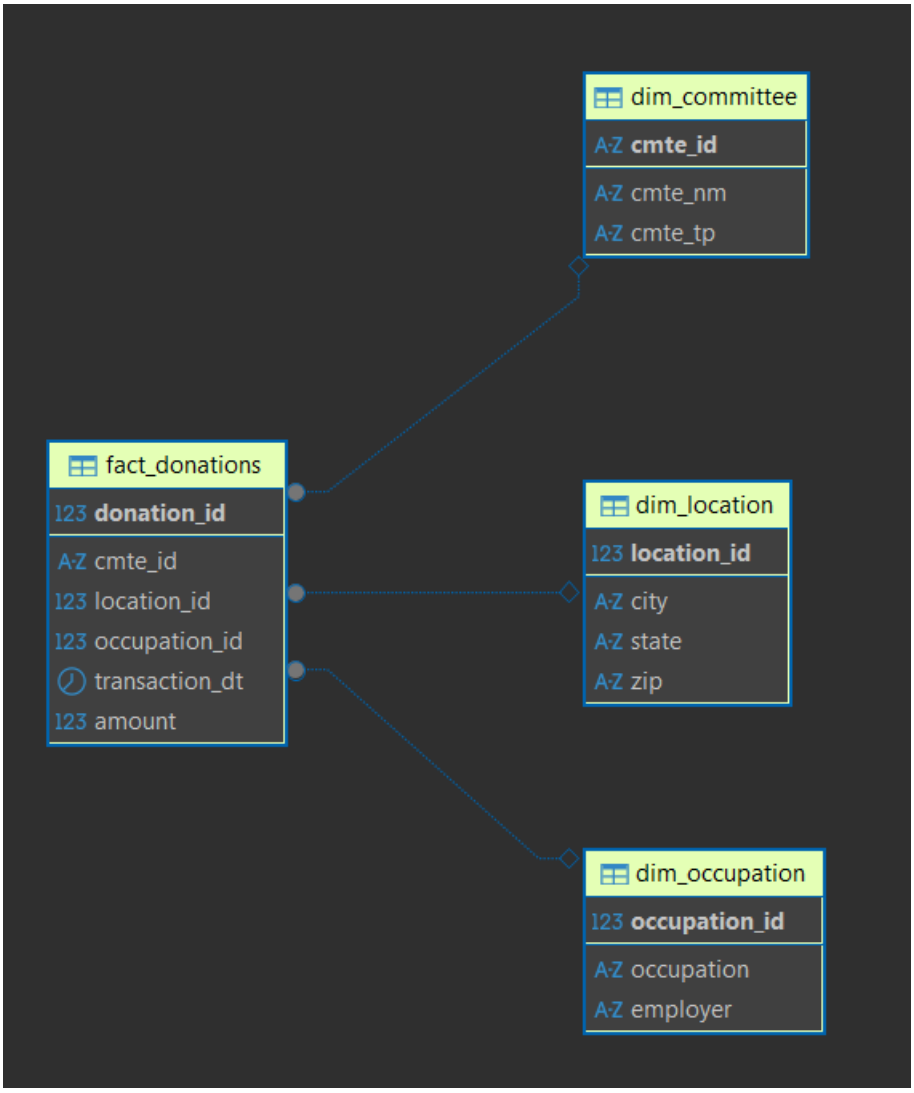
Tables schemas

File Features			
Description	This table describes the columns in the original datasets and gives a more complete name and a brief description of the data as it is not always obvious from the column name what it is referring to.		
Attribute	**Description**	**Type**	***Examples of values***
CMTE_ID	Committee ID	Characters	C00193433
AMNDT_IND	Amendment Indicator	Character	N
RPT_TP	Report type	Characters	M2
TRANSACTION_PGI	Page number	Character	P
IMAGE_NUM	FEC image ID	Integer	202302169578643000
TRANSACTION_TP	Transaction Type	Integer	15
ENTITY_TP	Entity Type	Characters	IND
NAME	Name	String	EPSTEIN, LYNN MS.
CITY	City	String	PALM BEACH
STATE	State	Characters	FL
ZIP_CODE	Zip Code	Integer	33480
EMPLOYER	Employer	String	NOT EMPLOYED
OCCUPATION	Occupation	String	RETIRED
TRANSACTION_DT	Date	Integer	1182023
TRANSACTION_AMT	Amount of contribution	Integer	2500
TRAN_ID	Transaction ID	Integer	7677450
FILE_NUM	FEC filing number	Integer	1689162
MEMO_CD	Memo code	Characters	“”
MEMO_TEXT	Memo text	Characters	“”
SUB_ID	Sequential ID	Integer	4021620231730020000
CMTE_ID_1	Committee ID	Characters	C00193433
cnt	counts	Integer	94361
CMTE_NM	Committee Name	String	EMILYS LIST
CMTE_TP	Committee Type	Character	Q

CMTE_DSGN	Committee Designation	Character	U
CMTE_FILING_FREQ	Filing Frequency	Character	M
TTL_RECEIPTS	Total receipts	Float	35013546.07
TRANS_FROM_AFF	Transfers	Float	0
INDV_CONTRIB	Total individual Contributions	Float	17506357.79
OTHER_POL_CMTE_CONTRIB	Contribution from PACs	Float	67750
TTL_DISB	Tot. disbursements	Float	34845664.21
TRANF_TO_AFF	Transfers to Affiliated PACs	Float	0
INDV_REFUNDS	Refunds to individuals	Float	63981.17
OTHER_POL_CMTE_REFUNDS	Refunds to other PACs	Float	0
CAND_LOAN_REPAY	Loan repayments to candidate	Float	0
LOAN_REPAY	Repayments of other loans	Float	0
COH_BOP	Cash on hand at beginning of period	Float	3619795.37
COH_COP	Cash on hand at close of period	Float	3787677.23
DEBTS_OWED_BY	Debts owed by committee	Float	0
NONFED_TRANS_RECEIVED	Non-federal funds received	Float	16321829.22
CONTRIB_TO_OTHER_CMTE	Contributions to other commitee	Float	646034.24
IND_EXP	Independent expenditures from committee	Float	0
PTY_COORD_EXP	Party coordinated expenses	Float	0
NONFED_SHARE_EXP	Share of expenses paid with non-federal funds	Float	1132857.6
CVG_END_DT	Coverage end date of period	Date	2/29/2024

Primary Key	Fact_id
Foreign Keys	CMTE_ID, date_id, contributor_id

Of these attributes, the numerical transaction type effectively acts as a foreign key to a table not included in the dataset so it will be excluded, same with report type. Transaction page, amendment indicator, other id, file number, memo code, and memo text are all irrelevant to the analysis and will also be excluded. Additionally, columns related to committee filing requirements will be excluded as it is not of interest to this analysis. However, the FEC ID will be retained as a reference to allow reference on external resources.



Above is the star schema of the data. Many of the dimensions from the original dataset are excluded as they are not of interest to study. Notably the names of individual donors are missing because they are not useful in the following analysis.

2.2 Hadoop Implementation

The input folder is in the Hadoop file system at user/root/termproject/input, and similarly the output folder is located at user/root/termproject/output. The raw "|"-delimited CSV files are included in the input folder and 4 tables were created in the output folder including 3 dimension tables and a fact table. The fact table consists of the following columns, location ID, occupation ID, committee ID, transaction date, amount, donor name, city, state, zip code, and the committee's FEC ID. The first 3 columns are keys to additional tables in the system that provide more information, such as while most of the location information is also included in the primary table, the raw data sometimes has the full ZIP code but it is truncated in the primary table for direct comparison. The full ZIP code is preserved in the location table. Similarly Occupation ID is used to reference a table containing both the occupation title, such as CEO as well as the corresponding business name. This format was constructed to not lose access to information that may be of interest in the analysis, but also to allow for the fact table to be the sole table used for analysis, and join operations to be performed if more information is needed from the dimension tables.

2.3 Reflective analysis of using a data warehouse vs Hadoop.

In reflection, the two systems perform very similar functions in allowing efficient access to large datasets. I was able to implement similar structures of fact and dimension tables in both systems. The main difference between using DBeaver for data warehousing and using Hadoop is that Hadoop is much better for performing operations on data and structuring it while DBeaver provides a useful GUI for the querying of databases, but much slower and more complicated to perform operations on the databases. DBeaver for data warehousing is just a more friendly way to use SQL/NoSQL commands while Hadoop provides a large toolbelt of Big Data operations and methods with it's powerful MapReduce for efficient and distributed storage. Overall I found Hadoop to be more intuitive and had a better experience but that may be due to my general unfamiliarity with SQL/NoSQL systems.

3. Data Preparation

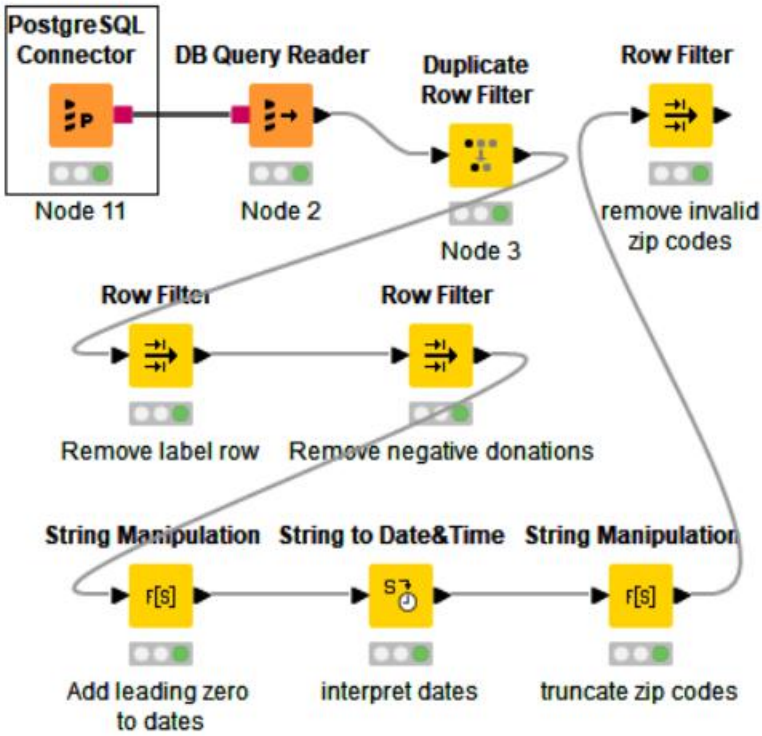
3.1 Relational Data Warehouse Implementation

For ETL considerations, the staging step consisted of locating the files which were to be analyzed from the provided drive, creating a blank table in DBeaver with the correct names of columns and data types to match the columns of the 12 csv files which were downloaded, and import 1-by-1 into the table ensuring no errors. This took several attempts to get the correct optimal data types for each column as some of the text-based columns were quite long mid-way through the files so the automatic detection would fail to create large enough data types.

In the extraction step, all attributes which do not contribute to either identifying information on the committee or contributor, or information on the contribution itself. This involved removing a large number of attributes to trim down to only several attributes which are anticipated to be used in the analysis.

For transformation steps, all missing values were ignored as they were not useful for the data analysis if they were missing, and 3 foreign keys were created to represent data which was redundant. An additional challenge was the [MMddyyyy][Mddyyyy] formatting of the dates, which ended up being easier to handle outside of DBeaver and then importing the formatted dates. Other selections for dimension tables are groupings of columns where to know 1 would be to know them all, such as knowing the committee's name would also tell you the type and designation of the committee, and knowing the name of the contributor would also tell you the city, state, zip code, etc. of the individual, so each of these was separated out into it's own table. These steps were largely done through SQL commands in DBeaver. The combined contributions table (combined table from all 12 csv files) took up 208M in disk space and had 258,637 rows. Meanwhile the fact table of the contributors only took up 87M in disk space and had 338,816 rows. The date table is much smaller at only 32K which is to be expected since there is a very limited number of days in the dataset, the contributor table has 46,115 rows identifying unique contributors and it takes up 5.4M in disk space and the committee table is very small with only 61 rows taking up 24K in disk space. This shows how the process of separating out tables causes a large reduction in disk space for the same data and makes it less costly for the computer to query.

When loading the data in KNIME for analysis it was very simple and I followed the steps from the Canvas tutorial. It allowed for easy creation of graphics to include mostly around the aggregation of the dollar amounts from the filings.



The above KNIME workflow is used to clean the data for the last step in the ETL process.

```
PS C:\Users\HP> docker exec -i -t 75876fda211a psql -U postgres -d termproject2
psql (12.22 (Ubuntu 12.22-3.pgdg24.04+1))
Type "help" for help.

termproject2=# \d fact_donations
                                        Table "public.fact_donations"
     Column      |      Type      | Collation | Nullable |                        Default
----------------+----------------+-----------+----------+-------------------------------------------------------
 donation_id    | bigint         |           | not null | nextval('fact_donations_donation_id_seq'::regclass)
 cmte_id        | text           |           |          |
 location_id    | integer        |           |          |
 occupation_id  | integer        |           |          |
 transaction_dt | date           |           |          |
 amount         | numeric(14,2)  |           |          |
Indexes:
    "fact_donations_pkey" PRIMARY KEY, btree (donation_id)
Foreign-key constraints:
    "fk_fact_cmte" FOREIGN KEY (cmte_id) REFERENCES dim_committee(cmte_id)
    "fk_fact_location" FOREIGN KEY (location_id) REFERENCES dim_location(location_id)
    "fk_fact_occupation" FOREIGN KEY (occupation_id) REFERENCES dim_occupation(occupation_id)
```

The above table is the result of running \d within the postgresql container on the fact table. Note the use of specific data types most relevant to the data being stored to optimize database performance/

3.2 Hadoop Implementation

For my analysis of the dataset in Hadoop I decided to use Pig for the ETL process. The process follows the 4 phases outlined in the lesson, namely staging, extraction, transformation, and loading.

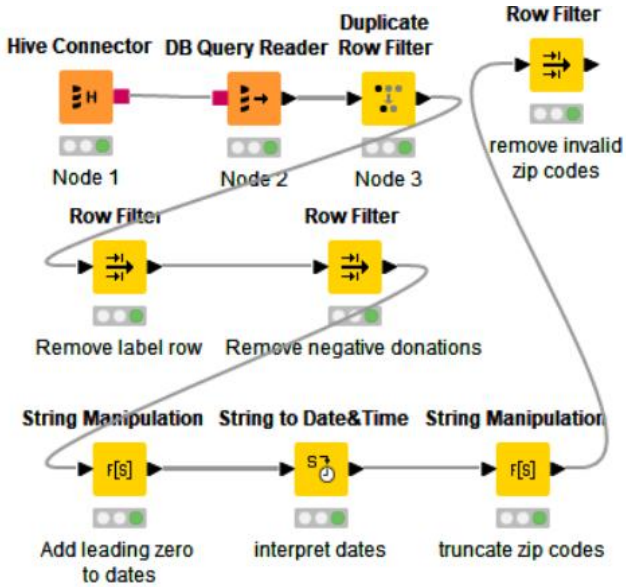
The staging step consisted of getting the 12 FEC CSV files into the HDFS system. They were put in a folder located at /user/root/termproject/input in my case.

For the extraction step, the raw data was loaded using Apache Pig using the "|" as the delimitator. For this step all columns are loaded and any column with any alphabetic data was loaded as a chararray and any column that was just numeric was loaded as a double.

Transformation of the data included cleaning and restructuring the data. This included removing empty lines and duplicate entries, filtering the columns to only be the columns of interest, and building dimension tables. These dimension tables consisted of a location dimension, an occupation dimension, and a committee dimension. Most of the excluded columns consisted of committee filing requirements which are not of interest in this analysis. The dimension tables were then joined to form a fact table to simplify analysis. Included in the fact table are keys for location, occupation, and committee, as well as areas of interest for analysis including transaction amount, date, and donor name.

Finally for the loading step the resulting tables were saved into a folder located at /user/root/termproject/output and then exported to a Hive table where it can be accessed in KNIME for report building. After ensuring duplicate rows were removed, the resulting table consists of 206,352 rows representing individual donations. This is significantly truncated in comparison to the Relational Data warehouse implementation, likely due to more robust filter techniques used in this implementation.

Additional work was done in KNIME to ensure clean data for analysis. Because the input data matches exactly, the loading process in KNIME also exactly matches the relational database version.



Above are the nodes used to prepare the data for analysis.

```
root@d2577d441e53:/# bl
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Connecting to jdbc:hive2://localhost:10000
Connected to: Apache Hive (version 3.1.2)
Driver: Hive JDBC (version 3.1.2)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.2 by Apache Hive
0: jdbc:hive2://localhost:10000> DESCRIBE fecdb.fact_transactions;
+-------------------+------------+----------+
|     col_name      | data_type  | comment  |
+-------------------+------------+----------+
| location_id       | int        |          |
| occupation_id     | int        |          |
| committee_id      | int        |          |
| transaction_date  | string     |          |
| amount            | double     |          |
| donor_name        | string     |          |
| city              | string     |          |
| state             | string     |          |
| zip               | string     |          |
| committee_fec_id  | string     |          |
+-------------------+------------+----------+
10 rows selected (0.243 seconds)
0: jdbc:hive2://localhost:10000>
```

Above is the apache hive data table used when connecting to KNIME.

3.3 Reflective analysis of data preparation in relational data warehouse vs Hadoop.

In reflection, preparing the data required much more SQL coding than I had anticipated from using DBeaver. While it is a powerful tool, it also has seemingly limited functionality without a reasonable ability to write and use SQL code. After I was able to get over that learning curve however, identifying the columns that needed to be separated out into different tables to form the fact and foreign tables was straightforward as it just took a little bit of looking at the data tables to see what information was redundant. I think the ERD visualization in DBeaver is very powerful and I think it is one of the greatest strengths of using that platform for data warehousing.

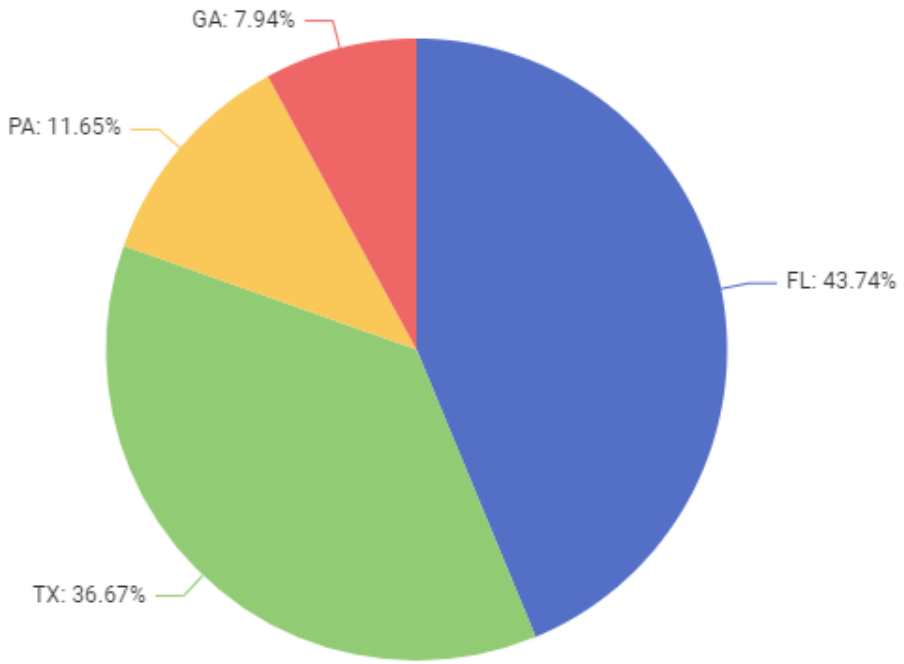
In comparison, Hadoop has many paths to get to the same goal. Whether using Apache Pig, Apache Spark, or Map Reduce, the same goal is achieved. None of these methods require the same ability to write SQL code as DBeaver and all exist in the command line rather than a GUI like DBeaver. This is both a strength and a weakness as it can be easier to find out how to do the transformation with any of the Hadoop methods, but the transformation in question may require a considerable amount of code, especially if using the Java based Map Reduce functions. Luckily Apache Pig is intuitive, and Apache Spark is just using Python which I am personally very familiar with. Overall, I ran into fewer issues implementing the ETL process in Hadoop compared to DBeaver, and I was impressed by the wide array of functionality provided by the platform. I also appreciated the strengths provided by the docker container in keeping the data partitioned.

4. Reporting System

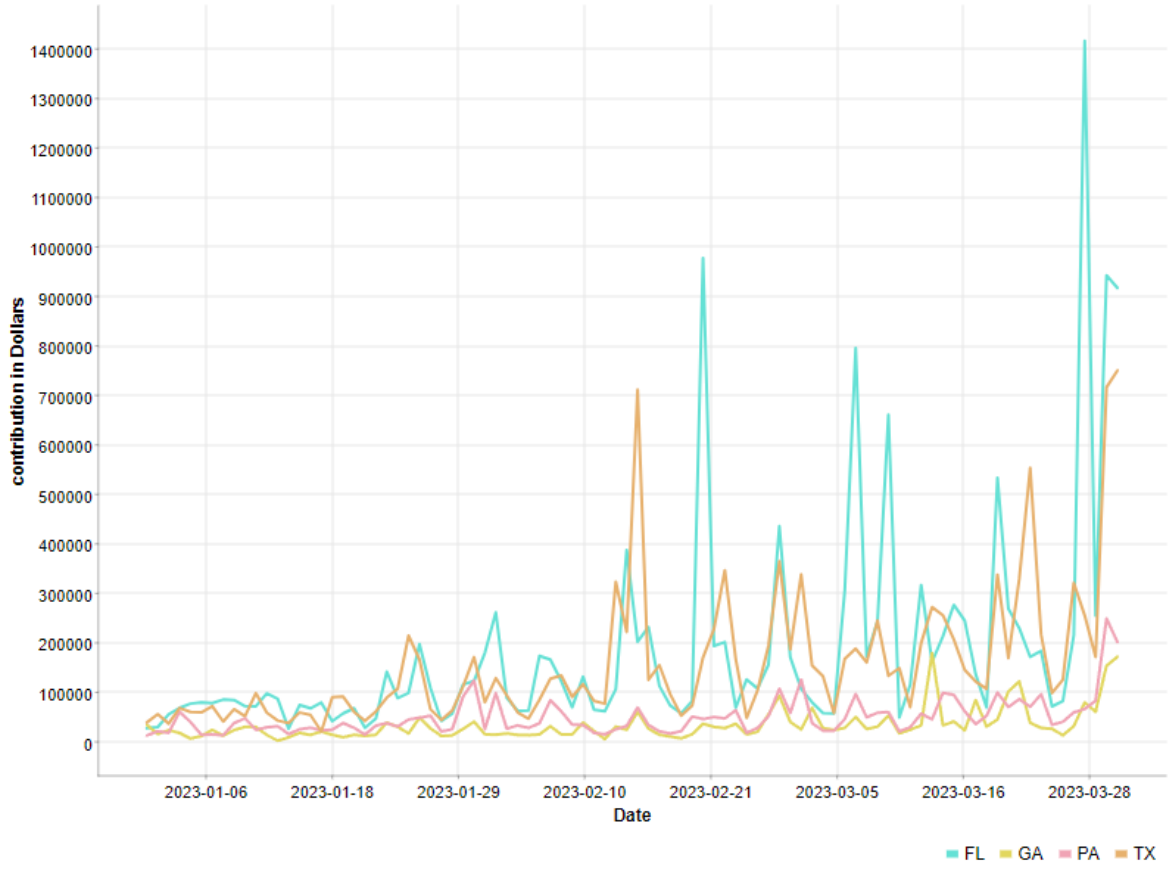
In this section I will share visualizations of the FEC data created in KNIME.

4.1 Relational Data Warehouse Implementation

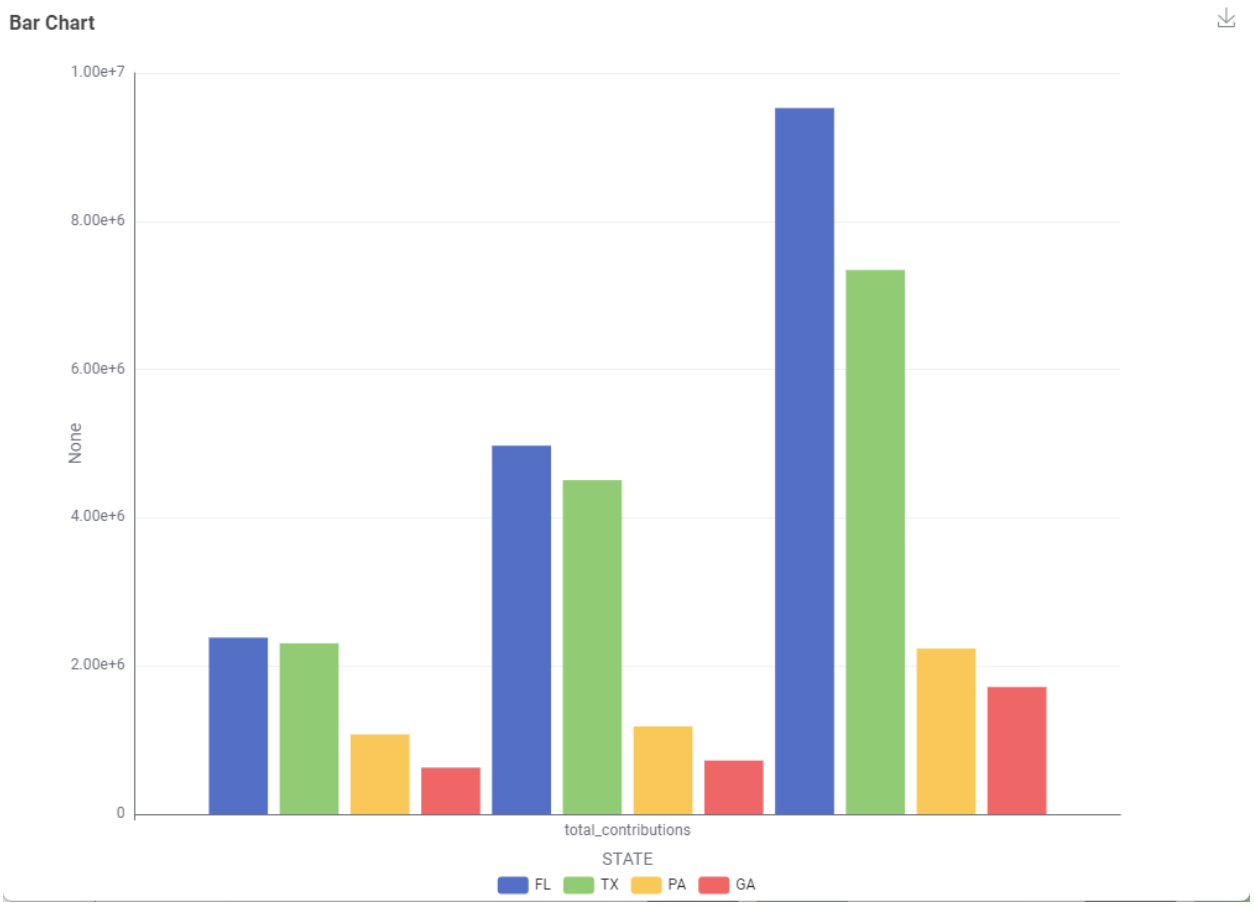
Pie Chart



This shows the relative contributions from each state across the entire data set. Interestingly Florida has most of the total by a large amount while only having 2/3 the population of Texas.



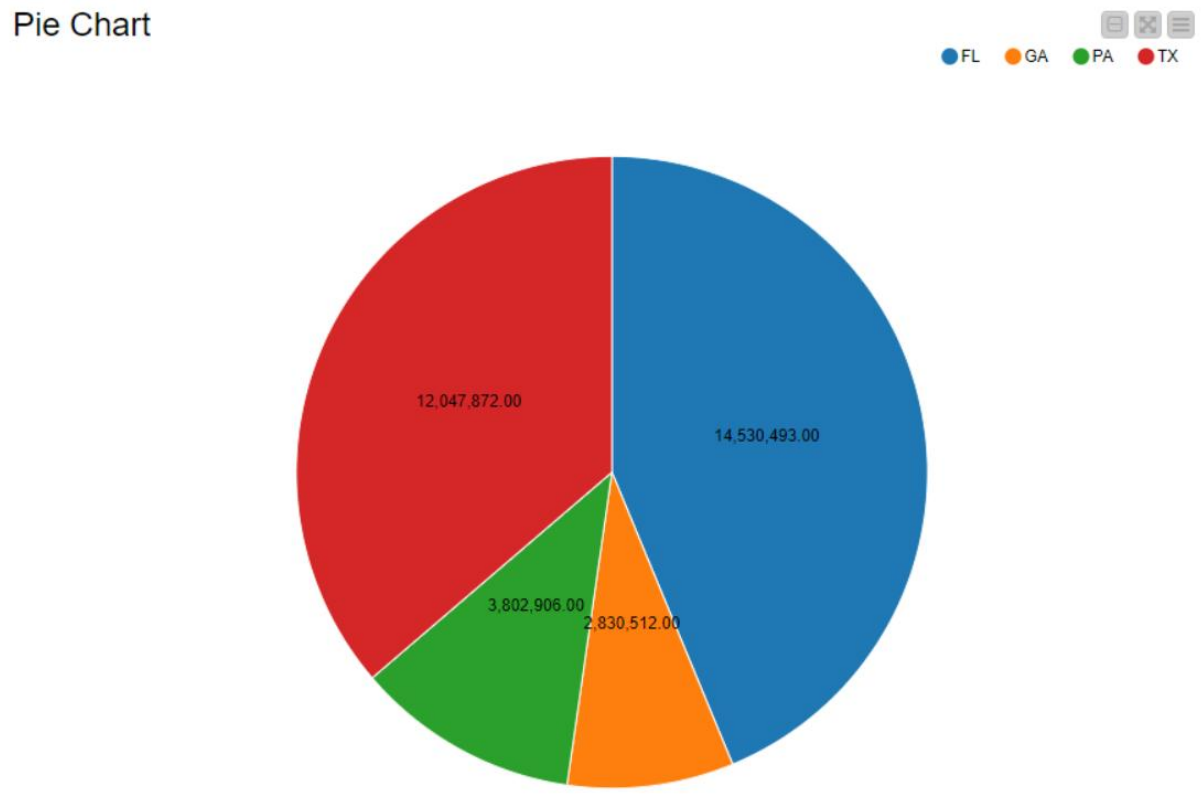
This visualization shows the contributions in each state over the time period in data. Interestingly the states all follow very similar trends, all starting low at the beginning of the year then ramping up drastically toward the end, with Florida getting especially large peaks in donations.



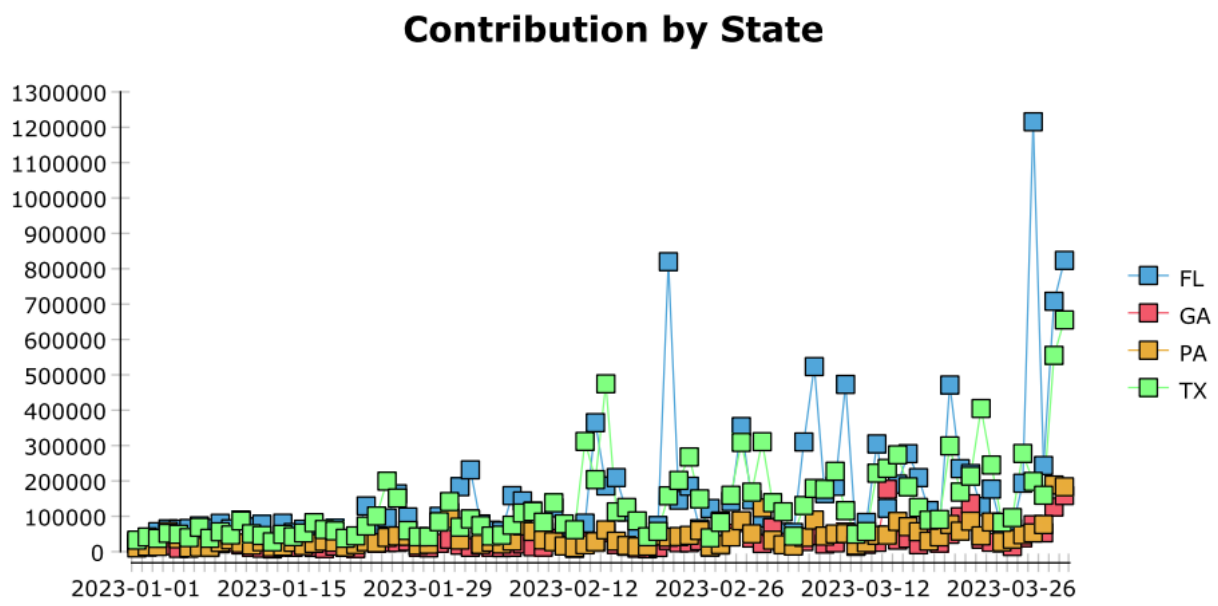
This visualization shows the net contributions by state in the 3 months of the dataset. This shows again that it was much more even at the beginning of the period and then Texas and Florida shoot ahead dramatically.

4.2 Hadoop Implementation

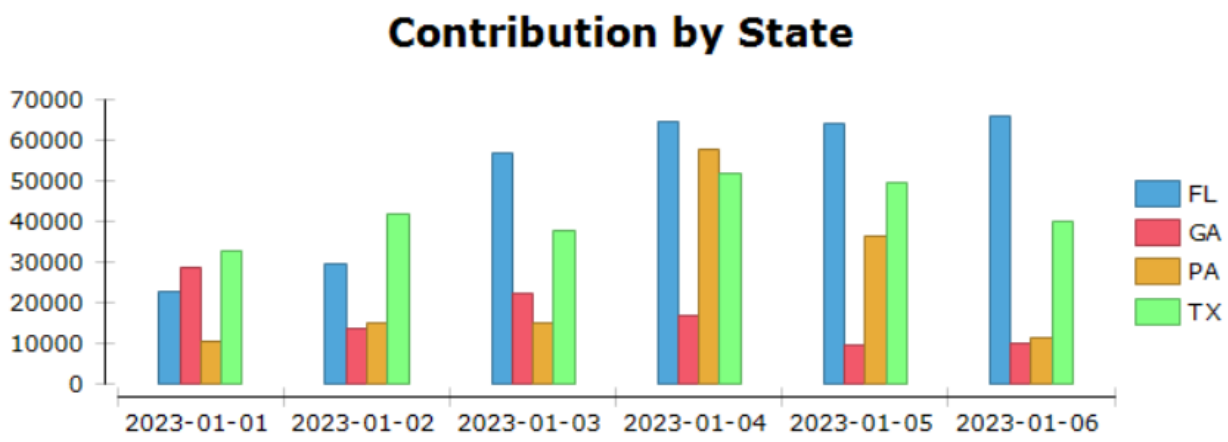
The version of KNIME I had to use for this was a bit older to get the functionality I wanted so the graphics look slightly different. This pie chart corresponds to the first pie chart from the other section and looks nearly identical.



This and the following chart were made in KNIME's report function with a data cube.



Once again it is created in the reporting functionality of KNIME.



4.3 Reflective analysis of result in relational data warehouse vs Hadoop.

In reflection for project 1, I can say that the filings are about what I expected since as it gets closer to election time we would expect contributions to ramp up. I would be interested to see even closer to election time how much more they increase. I chose these states as they are all fairly “purple” meaning there are more likely to be larger campaign donations and PAC donations and therefor more FEC filings for these states. I think it is interesting how even when accounting for difference in population Florida and Texas have much larger contributions in this dataset than PA or GA.

In reflection for part 2 of the project, I think there is no doubt that using Hadoop was much better in my experience than using a relational database. I would often need to wait a long time for things to load in DBeaver which was never the case when using Hadoop. It was also much more intuitive to use the HDFS system and was quite tidy too. I enjoyed learning the tools available with Hadoop and can easily imagine myself using it again in the future.

Conclusions

In conclusion, I think that while both have strengths, the Hadoop implementation is quite superior in my experience. I had fewer issues with Hadoop and spend significantly less time waiting for loading and scratching my head about trying to find a complex workaround for what I would consider to be a simple issue, such as interpreting the dates correctly. I admit it may be an issue of familiarity with the respective tools, but I do not see myself choosing DBeaver and PostGRES if given the choice.

An advantage of DBeaver is having a handy GUI that reduces the need to run commands to navigate the file system, while Hadoop seems to have a much larger toolbelt for handing common issues with ease. For disadvantages, DBeaver requires significant SQL coding in order to reasonably perform operations, while Hadoop has powerful tools like Apache Pig that while still are programming are much more forgiving to use.